

Audit Report

Table of Contents

1. Executive Summary

4

Executive Summary

4

Risk Profile

4

Overall Security Posture

4

Launch Recommendations

5

Audit Scope

5

2. Assumptions and Considerations

6

Assumptions & Considerations

6

Scope Limitations

6

3. Severity Definitions

7

Impact

7

Likelihood

7

Severity Classification Matrix

7

4. Findings

9

M01: TransceiverManager is Not Pausable

9

L01: NTTManager Doesn't Validate TransceiverInstruction Length

11

L02: Missing Validation Allows Thresholds Larger Than Number of Transceivers

12

5. Enhancement Opportunities

14

E01: Removing Redundant Check Will Improve Gas Efficiency

14

E02: Unused Variable total_fee in WormholeTransceiver._send_message

16

E03: Missing Zero Check in Message Approval Could Lead to Unvalidated Messages

18

E04: WormholeTransceiver Does Not Check Peer Chain ID Against Current Chain ID

19

E05: Unused Method _get_wormhole_emitter_sequence in WormholeTransceiver.py

20

E06: NttManager Should Be Pausable by the Owner in Addition to the Pauser

21

E07: Paused NttManager Should Not Be Unpausable by the Pauser Role

22

About Us

23

About Adevar Labs	23
Audit Methodology	23
Confidentiality Notice	24
Legal Disclaimer	24

1. Executive Summary

Executive Summary

About Folks Finance

Folks Finance is a decentralized finance (DeFi) protocol that has implemented Native Token Transfer (NTT) contracts on the Algorand blockchain. This audit focuses on the implementation of cross-chain token transfer mechanisms following the [Wormhole NTT Invariant](#) document.

Audit Summary

- Number of Findings: 3 total
- Critical: 0
- High:0
- Medium: 1
- Low: 2
- Number of Enhancement Opportunities: 7

Risk Profile

The following table summarizes the distribution of identified vulnerabilities by risk level:

Risk Level	Count	Fixed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	1	1	0
Low	2	0	2

Overall Security Posture

Before Fix Review

The Folks Finance team demonstrated a high level of expertise as no critical issues were found during this audit.

The NTT invariants have been correctly implemented in general, with a few discrepancies found compared to the invariants or reference implementation:

- Pausability mechanisms that do not fully align with the reference implementation
- Transceiver Instruction (INV-029)
- Validation threshold (INV-023)
- Chain ID check (INV-021)

After Fix Review

All discrepancies have been addressed:

- Pausability mechanisms are aligned as recommended
- Transceiver instruction length is already ensured by the Algorand maximum argument size limitation
- Validation threshold is already enforced by the current design
- Chain ID is checked

Launch Recommendations

Before launching the NTT contracts, the Folks Finance team should:

- Ensure complete alignment with invariants when applicable
- Review and align pausability mechanisms with the reference Wormhole implementation
- Add additional checks to prevent potential replay or unauthorized message execution

After Fix Review

Folks Finance correctly addressed each of the above launch recommendations.

Audit Scope

Repository: *Folks-Finance/algorand-ntt-contracts*

Commit Hash: **47bc4576903b9f37009159d14bac1a2ca2428456**

Files/Modules in Scope: *ntt_contracts/*

2. Assumptions and Considerations

Assumptions & Considerations

- The protocol implementation for Algorand does not aim to replicate 1:1 with the EVM reference implementation
- A method to notify the contract when receiving transfers was not considered necessary

Scope Limitations

Time-Limited Assessment: The security assessment was conducted over a fixed-time period. As such, not all potential security vulnerabilities may have been identified. Security assessment is a point-in-time exercise, and new vulnerabilities may emerge following the completion of this assessment.

Defined Scope: The assessment was limited to the systems, applications, and networks explicitly defined in the scope. Systems outside the defined scope, even if they interact with in-scope systems, were not assessed. A list of in-scope assets is provided in the Introduction section of this report.

3. Severity Definitions

Each issue identified in this report is assigned a severity level based on two dimensions: **Impact** and **Likelihood**. These dimensions help project our team's understanding of both the potential consequences of a vulnerability and how likely a vulnerability is to be discovered and exploited in the real world.

Impact

Impact reflects the potential consequences of the issue—particularly on **project funds, user funds**, and the **availability or integrity** of the protocol.

- **High Impact:** Successful exploitation could result in a complete loss of user or protocol funds, disruption of core protocol functionality, or permanent loss of control over critical components.
- **Medium Impact:** Exploitation could cause significant disruption or partial loss of funds, but not a total compromise. May impact some users or non-core functionality.
- **Low Impact:** The issue has minor or negligible consequences. It may affect edge cases, expose metadata, or degrade performance slightly without putting funds or core logic at serious risk.

Likelihood

Likelihood reflects how easy a vulnerability is to discover and exploit by an attacker, as well as how economically attractive the exploit is to an attacker.

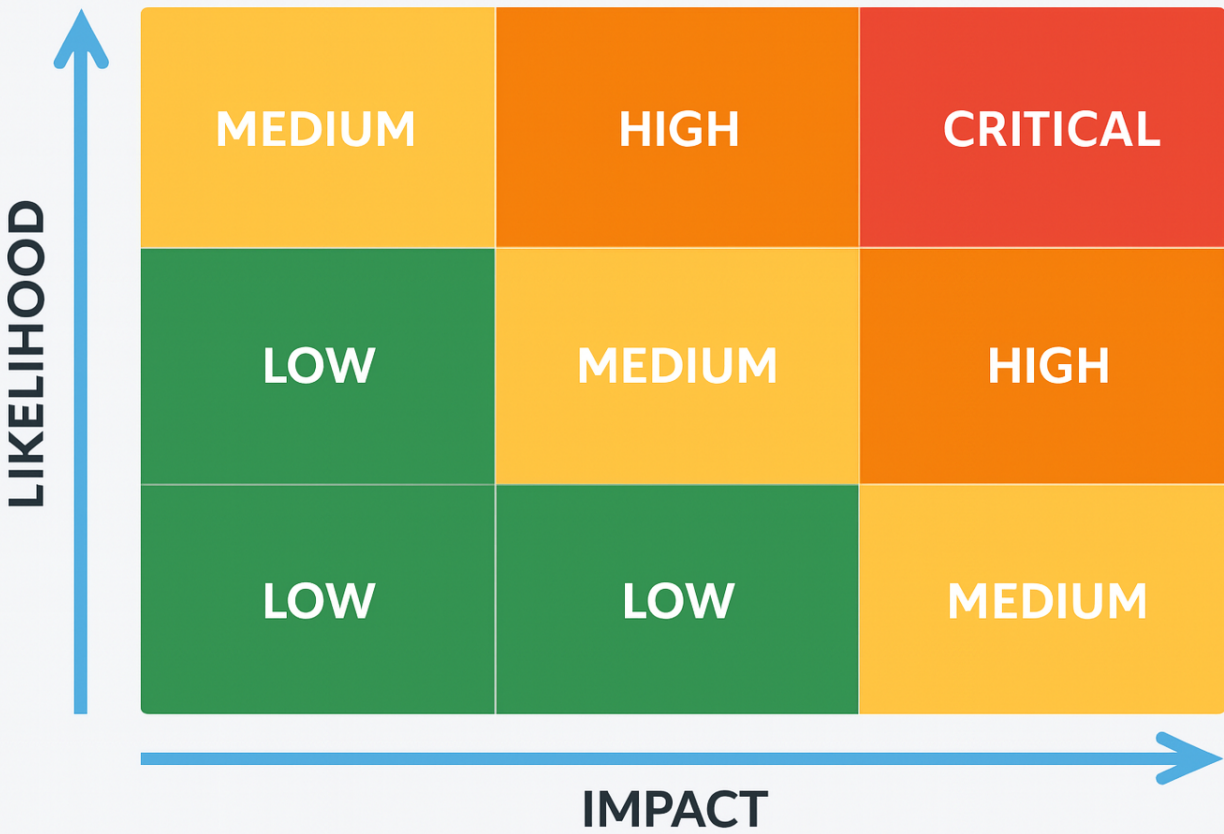
- **High Likelihood:** The vulnerability is trivially exploitable. This means it can be exploited by a wide range of actors without privileged access rights, with minimal capital requirements and low financial risks.
- **Medium Likelihood:** This type of vulnerability can be found and exploited with moderate effort. It might require a significant capital investment, but with manageable financial risk.
- **Low Likelihood:** Exploitation of these vulnerabilities is often technically unfeasible or requires highly specialized conditions. They may require extraordinary effort or a significant financial risk for an attacker, with a high chance of failure and minimal potential return.

Severity Classification Matrix

By combining **Impact** and **Likelihood**, we assign a severity level using the matrix below:

- **Critical:** High impact + high likelihood (e.g. a bug that could allow anyone to drain a substantial amount of protocol funds with minimal effort)
- **High:** High impact with medium likelihood, or medium impact with high likelihood
- **Medium:** Moderate impact and/or discoverability
- **Low:** Minimal impact or unlikely to be exploited

This structured approach helps teams prioritize fixes and mitigate the most dangerous threats first.



Severity Matrix

4. Findings

M01: TransceiverManager is Not Pausable

Status:

Resolved

Impact:



Likelihood:



Severity:

Medium

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacla2ca2428456/ntt_contracts/transceiver/TransceiverManager.py#L25-L25

>_ TransceiverManager.py

PYTHON

```
23:
24:
25: class TransceiverManager(ITransceiverManager, AccessControl):
26:     def __init__(self) -> None:
27:         AccessControl.__init__(self)
```

Description:

According to the Wormhole documentation and invariants, the protocol should be pausable in case of emergencies. However, there is a portion of the protocol that is not pausable. Specifically, the **TransceiverManager**, which in the Algorand implementation was decoupled from the **NTTManager**.

For example, in the EVM implementation, the **attestationReceived()** method is pausable in case of emergencies:

```
/// @inheritdoc INttManager
function attestationReceived(
    uint16 sourceChainId,
    bytes32 sourceNttManagerAddress,
    TransceiverStructs.NttManagerMessage memory payload
@>> ) external onlyTransceiver whenNotPaused {
```

JS

In the Algorand version, however, the **attestation_received()** method is implemented in the **TransceiverManager**, which is **not pausable**:

```
@abimethod
def attestation_received(self, message: MessageReceived) -> None:
```

PYTHON

Recommendation:

Make the **TransceiverManager** pausable, specifically the **attestationReceived()** method.

Developer Response:

Fixed.

Pause is added and checked for all interactions between Transceiver Manager and NttManager (**send_message_to_transceivers()** and **attestation_received()**)

L01: NTTManager Doesn't Validate TransceiverInstruction Length

Status:

Acknowledged

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacla2ca2428456/ntt_contracts/ntt_manager/NttManager.py#L272-L272

> _NttManager.py

PYTHON

```
270:
271:     @subroutine(inline=False)
272:     def _transfer_entry_point(
273:         self,
274:         fee_payment: gtxn.PaymentTransaction,
```

Description:

In the Wormhole NTT Specs, invariants, and other implementations (e.g., EVM), the length of the transceiver instruction is explicitly validated ([INV-029](#)) that must be implemented:

In the EVM version, both individual instructions and the complete array of instructions are validated for length:

```
function encodeTransceiverInstructions(
    TransceiverInstruction[] memory instructions
) public pure returns (bytes memory) {
    if (instructions.length > type(uint8).max) {
        revert PayloadTooLong(instructions.length);
    }
}

...

function encodeTransceiverInstruction(
    TransceiverInstruction memory instruction
) public pure returns (bytes memory) {
    if (instruction.payload.length > type(uint8).max) {
        revert PayloadTooLong(instruction.payload.length);
    }
}
```

SOLIDITY

Recommendation:

The NTTManager should validate the user-provided input parameter to conform to the specification by adding length checks for transceiver instructions.

Developer Response:

Acknowledged.

There is already a maximum argument size limitation on Algorand.

L02: Missing Validation Allows Thresholds Larger Than Number of Transceivers

Status:

Acknowledged

Impact:



Likelihood:



Severity:

Low

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacla2ca2428456/ntt_contracts/transceiver/TransceiverManager.py#L76-L88

>_ TransceiverManager.py

PYTHON

```
74:         emit(TransceiverAdded(ARC4UInt64(message_handler), ARC4UInt64(transceiver)))
75:
76:     @abimethod
77:     def remove_transceiver(self, message_handler: UInt64, transceiver: UInt64) -> None
78:     :
79:         self._check_message_handler_known(message_handler)
80:         self._check_sender_role(self.message_handler_admin_role(message_handler))
81:         # ensure transceiver was added
82:         old_transceivers = self.handler_transceivers[message_handler].copy()
83:         was_removed, new_transceivers = UInt64SetLib.remove_item(transceiver, old_transceivers)
84:         assert was_removed, "Transceiver was not added"
85:
86:         # remove transceiver
87:         self.handler_transceivers[message_handler] = new_transceivers.copy()
88:         emit(TransceiverRemoved(ARC4UInt64(message_handler), ARC4UInt64(transceiver)))
89:
90:     @abimethod(readonly=True)
```

Description:

- Invariant:** Attestation threshold must not exceed available transceivers and must be positive
- **Enforcement:** Explicit bounds checking in threshold setting functions
- **Error Codes:** **ThresholdTooHigh**, **ZeroThreshold** (EVM/Solana), **MISSING VALIDATION** (Sui)
- **Code Reference:** **_checkThresholdInvariants()** (EVM), error enforcement (Solana), **EThresholdTooHigh** (Sui)

In the method which removes transceivers, the threshold of the **MessageHandler** is not updated:

@abimethod

PY

```
def remove_transceiver(self, message_handler: UInt64, transceiver: UInt64) -> None:
    self._check_message_handler_known(message_handler)
    self._check_sender_role(self.message_handler_admin_role(message_handler))

    # ensure transceiver was added
    old_transceivers = self.handler_transceivers[message_handler].copy()
```

... continued

PY

```
        was_removed, new_transceivers = UInt64SetLib.remove_item(transceiver, old_transceivers
    )

    assert was_removed, "Transceiver was not added"

    # remove transceiver
    self.handler_transceivers[message_handler] = new_transceivers.copy()
    emit(TransceiverRemoved(ARC4UInt64(message_handler), ARC4UInt64(transceiver)))
```

Recommendation:

Automatically decrease threshold when removing a transceiver to ensure it does not exceed the number of available transceivers.

Developer Response:

Acknowledged.

This isn't possible because the configured transceivers is in the TransceiverManager and the set threshold is in the NttManager. Any blocked message executions will be temporary until the admin sets a lower threshold.

5. Enhancement Opportunities

E01: Removing Redundant Check Will Improve Gas Efficiency

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacl2ca2428456/ntt_contracts/transceiver/TransceiverManager.py#L134-L141

```
> _TransceiverManager.py PYTHON
132:         assert fee_payment.amount == total_delivery_price, "Incorrect fee payment"
133:
134:     @abimethod
135:     def attestation_received(self, message: MessageReceived) -> None:
136:         # check caller is configured transceiver for message handler
137:         transceiver = Global.caller_application_id
138:         assert transceiver, "Caller must be an application"
139:         message_handler = MathLib.safe_cast_uint256_to_uint64(UInt256.from_bytes(message.handler_address.bytes))
140:         self._check_message_handler_known(message_handler)
141:         self._check_transceiver_configured(message_handler, transceiver)
142:
143:         # calculate message digest and retrieve number of attestations
```

Description:

```
@abimethod PYTHON
def attestation_received(self, message: MessageReceived) -> None:
    # check caller is configured transceiver for message handler
    transceiver = Global.caller_application_id
    assert transceiver, "Caller must be an application"
    message_handler = MathLib.safe_cast_uint256_to_uint64(UInt256.from_bytes(message.handler_address.bytes))
    self._check_message_handler_known(message_handler)
    self._check_transceiver_configured(message_handler, transceiver)
```

There is a redundant check in the `attestation_received()` method. Specifically, `_check_message_handler_known()` is called before `_check_transceiver_configured()`, which internally calls the same function:

```
@subroutine PYTHON
def _check_transceiver_configured(self, message_handler: UInt64, transceiver: UInt64) -> None:
    assert self.is_transceiver_configured(message_handler, transceiver), "Transceiver not configured"
    ...

@abimethod(readonly=True)
def is_transceiver_configured(self, message_handler: UInt64, transceiver: UInt64) -> Bool:
    self._check_message_handler_known(message_handler)
    return UInt64SetLib.has_item(transceiver, self.handler_transceivers[message_handler].copy())
```

Potential Benefit:

Removing the redundant check will improve gas efficiency.

Recommendation:

Simply remove the redundant check to fix this:

@abimethod

DIFF

```
def attestation_received(self, message: MessageReceived) -> None:
    # check caller is configured transceiver for message handler
    transceiver = Global.caller_application_id
    assert transceiver, "Caller must be an application"
    message_handler =
    MathLib.safe_cast_uint256_to_uint64(UInt256.from_bytes(message.handler_address.bytes))
-    self._check_message_handler_known(message_handler)
    self._check_transceiver_configured(message_handler, transceiver)
```

Developer Response:

Fixed according to the recommendation.

E02: Unused Variable `total_fee` in `WormholeTransceiver._send_message`

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacla2ca2428456/ntt_contracts/transceiver/WormholeTransceiver.py#L190-L213

> `WormholeTransceiver.py`

PYTHON

```
188:
189:     @subroutine
190:     def _send_message(self, total_fee: UInt64, message: MessageToSend, transceiver_instruction: Bytes) -> None:
191:         self._only_initialised()
192:
193:         # check whether peer chain is registered
194:         self.get_wormhole_peer(message.destination_chain_id)
195:
196:         # transceiver_instruction is ignored, when automatic relay is supported
197:         # it could be used to signal the relay approach (automatic/manual)
198:         handler_payload = message.id.bytes + message.user_address.bytes + message.payload
199:         payload = (
200:             Bytes.from_hex(WH_TRANSCEIVER_PAYLOAD_PREFIX) +
201:             message.source_address.bytes +
202:             message.handler_address.bytes +
203:             UInt16(handler_payload.length).bytes +
204:             handler_payload +
205:             UInt16(0).bytes # transceiver payload empty
206:         )
207:
208:         # publish message
209:         wormhole_core_address, exists = op.AppParamsGet.app_address(self.wormhole_core.value)
210:         assert exists, "Wormhole Core address unknown"
211:
212:         core_fee = self._get_wormhole_core_message_fee()
213:         payment = itxn.Payment(receiver=wormhole_core_address, amount=core_fee, fee=0)
214:         app_call = itxn.ApplicationCall(
215:             app_id=self.wormhole_core.value,
```

Description:

`_send_message` is called from the `Transceiver.send_message` method, calculating the delivery fee using `_quote_delivery_price`. Then this calculated fee is ignored, and instead, the wormhole core fee is used.

Potential Benefit:

Reduce possibilities for bugs in the future where quoting the delivery fee might be different from just the wormhole core base fee. This also improves performance and reduces gas costs.

Recommendation:

Use `total_fee` to pay for the wormhole core message publishing. Remove the call to wormhole core fee calculation.

@subroutine

DIFF

```
def _send_message(self, total_fee: UInt64, message: MessageToSend, transceiver_instruction: Bytes) -> None:
    self._only_initialised()
```

... continued

DIFF

```
# check whether peer chain is registered
self.get_wormhole_peer(message.destination_chain_id)

# transceiver_instruction is ignored, when automatic relayer is supported
# it could be used to signal the relay approach (automatic/manual)
handler_payload = message.id.bytes + message.user_address.bytes + message.payload
payload = (
    Bytes.from_hex(WH_TRANSCEIVER_PAYLOAD_PREFIX) +
    message.source_address.bytes +
    message.handler_address.bytes +
    UInt16(handler_payload.length).bytes +
    handler_payload +
    UInt16(0).bytes # transceiver payload empty
)

# publish message
wormhole_core_address, exists = op.AppParamsGet.app_address(self.wormhole_core.value)
assert exists, "Wormhole Core address unknown"

-     core_fee = self._get_wormhole_core_message_fee()
-     payment = itxn.Payment(receiver=wormhole_core_address, amount=core_fee, fee=0)
payment = itxn.Payment(receiver=wormhole_core_address, amount=total_fee, fee=0)
app_call = itxn.ApplicationCall(
    app_id=self.wormhole_core.value,
    app_args=(Bytes(b"publishMessage"), payload, op.itob(0)),
    accounts=(Account(self.emitter_lsig.value.bytes),),
    fee=0,
)
itxn.submit_txns(payment, app_call)
```

Developer Response:

*Fee used is now the fee provided from parameters (**total_fee**), and **core_fee** has been removed.*

E03: Missing Zero Check in Message Approval Could Lead to Unvalidated Messages

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bac1a2ca2428456/ntt_contracts/transceiver/MessageHandler.py#L77

> **MessageHandler.py**

PYTHON

```
75:         fee=0,  
76:     )  
77:     return Bool(message_attestations >= self.threshold.value)  
78:  
79:     @abimethod(readonly=True)
```

Description:

In **MessageHandler.is_message_approved**, a message is considered approved if its number of attestations is equal to or larger than the threshold. If the threshold is set to 0, this would mean that any message could be considered approved. Currently setting the threshold to 0 is not possible, but if through further changes in the code this becomes possible, not having this check could be a significant vulnerability.

Potential Benefit:

Ensures that every approved message has at least one valid attestation.

Recommendation:

Add a check to verify that the number of attestations is larger than 0 as well.

Developer Response:

Fixed according to the recommendation.

E04: WormholeTransceiver Does Not Check Peer Chain ID Against Current Chain ID

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14ba2ca2428456/ntt_contracts/transceiver/WormholeTransceiver.py#L72-L84

> **WormholeTransceiver.py**

PYTHON

```
70:
71:     @abimethod
72:     def set_wormhole_peer(self, peer_chain_id: UInt16, peer_contract_address: Bytes32)
       -> None:
73:         """Set the WormholeTransceiver on a peer chain, overriding if needed.
74:
75:         Args:
76:             peer_chain_id: The peer chain to set
77:             peer_contract_address: The peer contract address
78:         """
79:         self._only_initialised()
80:         self._check_sender_role(self.manager_role())
81:
82:         # set peer (overriding if needed)
83:         self.wormhole_peers[peer_chain_id] = peer_contract_address.copy()
84:         emit(WormholePeerSet(peer_chain_id, peer_contract_address))
85:
86:     @abimethod
```

Description:

In the method **WormholeTransceiver.set_wormhole_peer**, there should be a check to ensure that the current chain will not be registered as a peer, similar to the check in **NttManager**. This is a violation of the INV-021 invariant.

Potential Benefit:

Prevents violating the peer management invariant by explicitly disallowing self-registration.

Recommendation:

Add an assertion to check if the **peer_chain_id** is the same as the current chain ID.

Developer Response:

Fixed according to the recommendation.

E05: Unused Method `_get_wormhole_emitter_sequence` in `WormholeTransceiver.py`

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacl2ca2428456/ntt_contracts/transceiver/WormholeTransceiver.py#L229-L236

> `WormholeTransceiver.py`

PYTHON

```
227:
228:     @subroutine
229:     def _get_wormhole_emitter_sequence(self) -> UInt64:
230:         sequence, exists = op.AppLocal.get_ex_bytes(
231:             Account(self.emitter_lsig.value.bytes),
232:             self.wormhole_core.value,
233:             Bytes.from_hex("00")
234:         )
235:         assert exists, "Sequence is unknown"
236:         return op.extract_uint64(sequence, 0)
237:
238:     @subroutine
```

Description:

The method `_get_wormhole_emitter_sequence` in `WormholeTransceiver.py` is not used anywhere in the code base.

Potential Benefit:

Removing this method will reduce the minimum balance slightly, but more importantly, it could be that this method is needed somewhere and its usage was forgotten.

Recommendation:

Delete the method or investigate why it exists.

Developer Response:

`_get_wormhole_emitter_sequence()` method was removed.

E06: NttManager Should Be Pausable by the Owner in Addition to the Pauser

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacl2ca2428456/ntt_contracts/ntt_manager/NttManager.py#L98-L104

> **NttManager.py**

PYTHON

```
96:
97:     @abimethod
98:     def pause(self, is_paused: Bool) -> None:
99:         self._only_initialised()
100:         self._check_sender_role(self.pauser_role())
101:
102:         # set whether paused or not
103:         self.is_paused = is_paused.native
104:         emit(Paused(is_paused))
105:
106:     @abimethod
```

Description:

The Owner/Admin (as well as the Pauser) should be able to pause the NttManager as in the reference implementation.
(Roles explanation from Wormhole: [video](#))

Potential Benefit:

Closer to the reference implementation and provides the Owner (Admin) full control of the contract as in the reference implementation.

Recommendation:

Split the pause method into **pause** and **unpause**. Make **pause** callable by the Pauser and Admin role, and **unpause** only by the Admin. (As in the reference implementation ManagerBase.sol)

Developer Response:

The NttManager uses role-based access control which allows us to assign a role to multiple accounts. This is different from the EVM implementation where only one address can be the “owner” and only one address can be the “pauser”. Therefore you can simply assign the “PAUSER” role to the same addresses that the “NTT_MANAGER_ADMIN” role is assigned to.

E07: Paused NttManager Should Not Be Unpausable by the Pauser Role

Location: https://github.com/Folks-Finance/algorand-ntt-contracts/blob/47bc4576903b9f37009159d14bacla2ca2428456/ntt_contracts/ntt_manager/NttManager.py#L98-L104

> **NttManager.py**

PYTHON

```
96:
97:     @abimethod
98:     def pause(self, is_paused: Bool) -> None:
99:         self._only_initialised()
100:         self._check_sender_role(self.pauser_role())
101:
102:         # set whether paused or not
103:         self.is_paused = is_paused.native
104:         emit(Paused(is_paused))
105:
106:     @abimethod
```

Description:

The **NttManager.py** contract can be unpaused by the Pauser, which doesn't match the reference EVM implementation where the Pauser can only pause the contract, and the owner has the exclusive right to unpause. (Roles explanation: <https://www.youtube.com/watch?v=Od5cTaxjTiw&t=264s>)

More users are meant to have access to the Pauser role compared to the Admin role, so that in an event of an exploit, users can react faster (with no governance process) and pause the contract to prevent loss of funds.

Since access to the Pauser is more relaxed, a malicious user can also gain access to this role more easily and then unpause a previously paused contract and proceed to exploit it.

Potential Benefit:

Restricting the unpause functionality to the Admin/Owner role ensures that the broader Pauser can act quickly to stop operations during emergencies without introducing the risk that they could later resume operations maliciously or prematurely.

Recommendation:

Split the pause method into **pause** and **unpause**. Make **pause** callable by the Pauser and Admin role, and **unpause** only by the Admin. (As in the reference implementation `ManagerBase.sol`)

Developer Response:

"UNPAUSER" role was added for both NTTManager and TransceiverManager.

About Us

About Adevar Labs

Adevar Labs is a boutique blockchain security firm specializing in web3 audits.

Built by a mix of experienced professionals in traditional enterprise and crypto natives who have contributed to some of the most critical projects in blockchain infrastructure.

Our team's background spans companies like Bitdefender, Asymmetric Research, Quantstamp, Chainproof, and Juicebox, and includes experience securing smart contracts, bridges, and L1 and L2 protocols across ecosystems like Solana, Ethereum, Polkadot, Cosmos, and MultiversX.

With over 100 audits completed and a portfolio that includes custom fuzzers, exploit modeling, and runtime testing frameworks, Adevar Labs brings both depth and precision to every engagement.

Our auditors have discovered critical vulnerabilities, built high-impact tooling, and placed in top positions in premier audit competitions including Code4rena and Sherlock.

Team members hold distinctions such as PhDs in software protection, and key roles at Fortune 500 companies, and leadership of flagship conferences like ETH Bucharest.

With team members having publications with over 1,100 academic citations and trusted by projects securing over \$500M in on-chain value, Adevar Labs blends elite technical rigor with real-world security impact.

We also collaborate with some of the best independent security researchers in the web3 space.

Projects may optionally request specific contributors to be part of their audit.

Audit Methodology

Our audit methodology is specialized to provide thorough security assessments of Algorand protocols. We focus explicitly on rigorous manual analysis, detailed threat modeling, and careful validation of implemented fixes to ensure your contracts operate securely and as intended.

1. Program Context and Architecture Analysis

Our auditors begin by deeply examining your Algorand contract's documentation, intended functionality, and state storage design (both global and local). We meticulously map out application call interactions, Algorand Virtual Machine (AVM) execution flows, and state management logic. Special attention is given to understanding how your contract interfaces with critical Algorand features, such as ASA (Algorand Standard Asset) management, inner transaction composition, and application account rekeying. Last but not least we check external integrations with other Algorand projects and verify if the inputs and return values are handled properly.

2. Threat Modeling

We conduct targeted threat modeling tailored specifically for the Algorand execution environment. We carefully define attacker capabilities and identify potential vulnerabilities that may arise from Algorand-specific issues, including but not limited to:

- Improper authorization checks, sender verification, or role assignment
- Misuse of Application Binary Interface (ABI) arguments and state-modifying calls

- Incorrect use of inner transactions or transaction group constraints
- Failure to adequately manage Global or Local State permissions and updates
- Logic flaws in Box storage access, capacity, and lifecycle management
- Arithmetic and integer handling risks within TEAL logic
- Risks stemming from asset opt-in and transfer logic

3. In-depth Manual Security Review

Our experienced auditors perform an extensive manual security review of your PyTeal/TEAL-based Algorand smart contracts. This involves a comprehensive line-by-line inspection of source code, focusing on common Algorand vulnerabilities including, but not limited to:

- Missing or insufficient sender/signer checks or transaction group checks
- Inadequate validation of account or asset parameters passed in the foreign arrays
- Incorrect logic in inner transaction construction (e.g., fee payment, receiver checks)
- Arithmetic and integer overflow or underflow errors (e.g., improper handling of btoi outputs)
- Unsafe use of opcodes or improper handling of TEAL execution limits
- Logic flaws in financial operations or ASA issuance/transfer/clawback state transitions
- Edge-case handling in ABI method argument validation and application state reads/writes
- Potential denial-of-service vectors related to transaction resource exhaustion or state limits

During this stage, we clearly document any discovered vulnerabilities, including detailed descriptions, precise severity ratings, and recommendations for secure implementation. In situations where it is not clear how the vulnerability might be exploited we may also include a detailed proof-of-concept exploit including code snippets and instructions on how the exploit could be performed.

4. Detailed Fix Review and Validation

After the initial audit and your team's subsequent remediation efforts, we perform a comprehensive fix review to ensure the vulnerabilities identified have been effectively resolved.

We verify each fix individually, confirming that:

- Corrections effectively eliminate the security risks
- Changes do not inadvertently introduce new vulnerabilities or regressions
- The fixes align closely with Algorand best practices and secure coding guidelines

Our detailed validation ensures that security improvements are robust, complete, and aligned with best practices specific to the Algorand development ecosystem.

Confidentiality Notice

This report, including its content, data, and underlying methodologies, is subject to the confidentiality and feedback provisions in your agreement with Adevar Labs. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Adevar Labs.

Legal Disclaimer

The review and this report are provided by Adevar Labs on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Adevar Labs disclaims all warranties, expressed or implied, in connection with this report, its content, and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

You agree that access to and/or use of the report and other results of the review, including any associated services, products, protocols, platforms, content, and materials, will be at your sole risk.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided.

You acknowledge that blockchain technology remains under development and is subject to unknown risks and flaws. Adevar Labs does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open-source or third-party software, code, libraries, materials, or information accessible through the report. As with the purchase or use of a product or service in any environment, you should use your best judgment and exercise caution where appropriate.